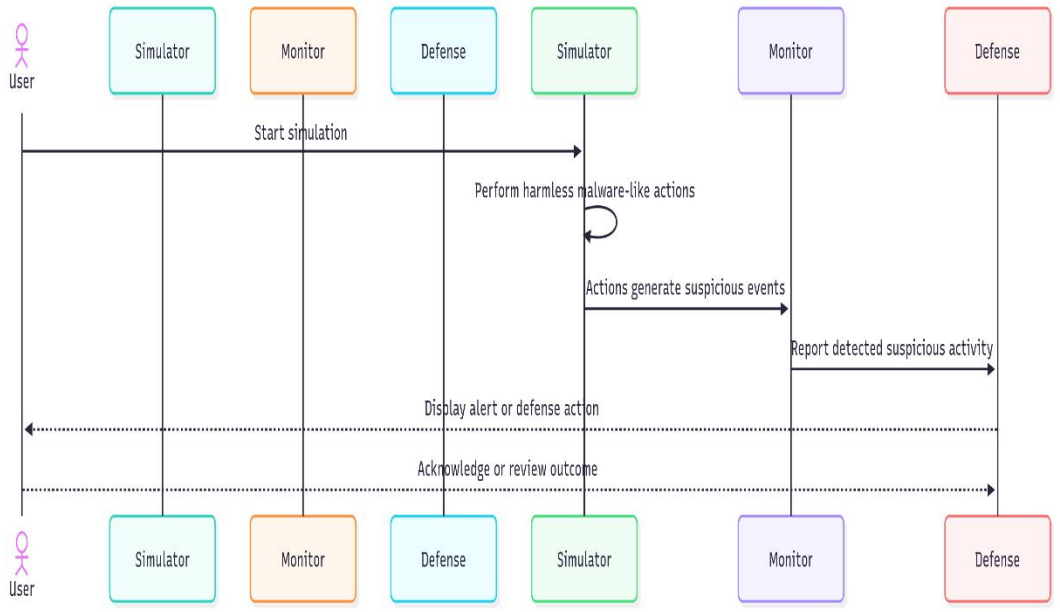
DEPARTMENT	**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**				
Semester: VII	**Course Title: Data Security & Privacy Lab**			**Course Code: 22AI73**	
PROGRAM QUESTION	Build a virus simulation tool — safe, educational, and non-malicious — to help you understand how malware might behave in a system, and how defenders can detect or prevent it.				
STUDENT NAMES	Udayaraj		USN	1DS22AI057	
LAB MARKS	**Program Execution (10)**	**Source Code (10)**	**Result (5)**	**Innovation/ Extra Features (5)**	**Total**
TOOLS USED	Cursor				
MAP COURSE OUTCOME	CO1				
MAP PROGRAM OUTCOME	PO1,PO2, PO3, PO4, PO5				
MAP PROGRAM SPECIFIC OUTCOME	PSO1, PSO2				
UML / SEQUENCE DIGRAM					
COURSE COORDINATOR	Dr ARUNA M G & Prof ENSTEIH SILVIA				

Code for Program 3:

Build a virus simulation tool — safe, educational, and non-malicious — to help you understand how malware might behave in a system, and how defenders can detect or prevent it.

```
import streamlit as st, networkx as nx, random, pandas as pd
from dataclasses import dataclass, field

S,I,P,Q = "Susceptible","Infected","Patched","Quarantined"
class Simulation:
    G: nx.Graph; node_attrs: dict = field(default_factory=dict); time_step: int = 0; history: list = field(default_factory=list)
    def reset(self, inf, seed):
        random.seed(seed)
        self.node_attrs = {n:S for n in self.G.nodes()}
        for s in random.sample(list(self.G.nodes()), min(inf, self.G.number_of_nodes())): self.node_attrs[s]=I
        self.time_step, self.history = 0, [self.snapshot()]
    def snapshot(self): return {"t": self.time_step, **{k:self.node_attrs.values().count(k) for k in [S,I,P,Q]}}

def step(sim, pt, pp, pq):
    sim.time_step +=1; new={}
    for n,s in sim.node_attrs.items():
        if s==I:
            new.update({nb:I for nb in sim.G.neighbors(n) if sim.node_attrs[nb]==S and random.random()<pt})
            if random.random()<pq: new[n]=Q
        elif s==S and random.random()<pp: new[n]=P
    sim.node_attrs.update(new); sim.history.append(sim.snapshot()); return sim

st.title(" Malware Simulator")
topo,n,inf,pt,pp,pq,seed = st.selectbox("Topology", ["Random","Scale-free"]), st.slider("Nodes",10,100,30), st.slider("Init Infected",1,10,2), st.slider("Transmit Prob.",0.0,1.0,0.1), st.slider("Patch Prob.",0.0,0.5,0.05), st.slider("Quarantine Prob.",0.0,1.0,0.5), st.number_input("Seed",0,999999,42)
G = nx.erdos_renyi_graph(n,0.1,seed) if topo=="Random" else nx.barabasi_albert_graph(n,2,seed)

if "sim" not in st.session_state or st.button("Reset"):
    sim=Simulation(G); sim.reset(inf,seed); st.session_state.sim=sim
else: sim=st.session_state.sim

if st.button("Step"):
    sim=step(sim,pt,pp,pq); st.session_state.sim=sim

st.line_chart(pd.DataFrame(sim.history).set_index("t")[[S,I,P,Q]])
st.dataframe(pd.DataFrame({"Node": list(sim.node_attrs.keys()), "State": list(sim.node_attrs.values())}))
```

RESULT: